

System Overview

The system represents the event notification and real-time user interface rendering subsystem of the Hospital Management System, designed using the **Observer Design Pattern**. The core design goal is to decouple the central hospital database and business logic from the various graphical user interface (Java Swing) sub-panels.

The system is structured hierarchically where:

- **HospitalData** acts as the central **Subject (ConcreteSubject)** that maintains the primary data repositories and broadcasts state modifications.
- **HospitalObserver** provides a unified interface defining how independent viewing components intercept messages.
- Five distinct UI panels act as **Concrete Observers** that dynamically adapt, draw, and flush visual interfaces based on specific event signals passed through runtime execution paths.

This design improves:

- **Flexibility:** Views can subscribe and unsubscribe dynamically at runtime to conserve memory and execution threads.
 - **Scalability:** New reporting views, statistical cards, or notification listeners can be plugged in seamlessly without altering core business rules.
 - **Maintainability:** Isolates data logic from presentation logic, aligning directly with the Single Responsibility Principle (SRP) and Open/Closed Principle (OCP).
-

2. Class Descriptions

HospitalObserver (Interface)

Represents the common lifecycle interface for all event-driven layout panels in the system. It establishes the contract required to listen to state modifications from the subject core.

- **update(HospitalEvent event):** Callback routine executed dynamically to trigger independent re-rendering cycles.
- **subscribe():** Invoked by a view component to attach its reference pointer to the subject register.

- **unsubscribe():** Formally removes the component pointer to release resources and stop redundant background tasks.

HospitalData (ConcreteSubject)

The authoritative single source of truth containing patient files, personnel matrices, and logs. It processes business execution mutations and notifies system components.

- **Responsibilities include:** Appending or deleting components, managing central notification dispatches via loop traversals, and supplying getter parameters for pull-model updates.

DashboardObserver (Concrete Observer 1)

Represents the real-time statistical overview counter card wrapper.

- **Responsibilities include:** Intercepting additions or structural deletions to count total patients, determine active healthcare loads, and isolate specific clinical staff roles (e.g., matching instances of Doctor).

PatientTableObserver (Concrete Observer 2)

Represents the visual table grid subsystem tracking hospital patient logs.

- **Responsibilities include:** Flashing table entries completely upon patient events and mapping domain properties directly into Java Swing tabular view vectors.

StaffTableObserver (Concrete Observer 3)

Represents the employee data administration viewer panel.

- **Responsibilities include:** Rendering comprehensive employee registries, evaluating specific subtype details (e.g., doctor specializations or nurse shifts), and displaying polymorphically computed salaries.

DeptTreeObserver (Concrete Observer 4)

Represents the hierarchical organizational structure viewer.

- **Responsibilities include:** Walking through nested department hierarchies and structural patterns to construct a multi-level JTree tracking aggregates like total department payroll.

EventLogObserver (Concrete Observer 5)

Represents the live analytical diagnostic panel and logging dashboard.

- **Responsibilities include:** Accumulating chronological event histories, providing colored system badges, rebuilding history components from the central logs, and handling panel clears.